



## Rapport de Projet

# Étude Comparative d'Architectures Distribuées Clé-Valeur

Architecture Centralisée · DHT Non Structurée · DHT Structurée : Chord

---

Master Systèmes et Applications Répartis (SAR)

PSAR – Sorbonne Université

Encadrant : Prof. Alessio Pagliari

---

### Rédigé par :

Cedric Geneviève · Safa Gunes · Somaya Boudalil

---

Mai 2026

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Trois générations d'architectures de stockage distribué</b>                | <b>2</b>  |
| 1.1      | Étude des approches P2P structurée: Les DHTs                                  | 2         |
| 1.1.1    | Introduction au problème du Lookup                                            | 2         |
| 1.1.2    | Comparaison des stratégies de recherche                                       | 3         |
| 1.1.3    | Fondamentaux des Distributed Hash Tables                                      | 5         |
| 1.1.4    | Mécanismes des DHTs                                                           | 6         |
| 1.1.5    | Interfaces des DHTs                                                           | 6         |
| 1.1.6    | Récapitulatif                                                                 | 7         |
| 1.2      | Conclusion et fil conducteur du projet                                        | 7         |
| <b>2</b> | <b>Première génération : architecture centralisée inspirée de Napster</b>     | <b>8</b>  |
| 2.1      | Vue d'ensemble de l'architecture                                              | 8         |
| 2.2      | Fonctionnement des opérations fondamentales                                   | 8         |
| 2.2.1    | Opération PUT                                                                 | 8         |
| 2.2.2    | Opération GET                                                                 | 9         |
| 2.2.3    | Opération Delete                                                              | 9         |
| 2.3      | Gestion dynamique des nœuds                                                   | 10        |
| 2.3.1    | Intégration d'un nouveau nœud (JOIN)                                          | 10        |
| 2.3.2    | Retrait d'un nœud (LEAVE)                                                     | 10        |
| 2.3.3    | Détection des pannes par heartbeat                                            | 11        |
| 2.4      | Avantages et limites de l'approche centralisée                                | 11        |
| <b>3</b> | <b>Deuxième génération : architecture non structurée inspirée de Gnutella</b> | <b>11</b> |
| 3.1      | Vue d'ensemble de l'architecture                                              | 11        |
| 3.2      | Principe de fonctionnement                                                    | 12        |
| 3.3      | Opération PUT                                                                 | 13        |
| 3.4      | Opération GET et routage par flooding                                         | 14        |
| 3.5      | Gestion dynamique des nœuds                                                   | 15        |
| 3.5.1    | Intégration d'un nouveau nœud (JOIN)                                          | 15        |
| 3.5.2    | Retrait d'un nœud (LEAVE)                                                     | 16        |
| 3.5.3    | Détection des pannes et stabilisation                                         | 16        |
| 3.6      | Reverse path caching                                                          | 17        |
| 3.7      | Avantages et limites de l'approche non structurée                             | 18        |
| <b>4</b> | <b>Troisième génération : DHT structurée inspirée de Chord</b>                | <b>18</b> |
| 4.1      | Vue d'ensemble de l'architecture                                              | 18        |
| 4.2      | Principe de fonctionnement                                                    | 18        |
| 4.2.1    | Routage efficace : Finger Table                                               | 19        |
| 4.2.2    | Opération PUT                                                                 | 19        |
| 4.2.3    | Opération GET                                                                 | 19        |
| 4.3      | Gestion dynamique des nœuds                                                   | 20        |
| 4.3.1    | Intégration d'un nouveau nœud (JOIN)                                          | 20        |
| 4.3.2    | Retrait d'un nœud (LEAVE)                                                     | 20        |
| 4.3.3    | Détection des pannes et stabilisation                                         | 20        |
| 4.4      | Avantages et limites de l'approche Chord                                      | 20        |
| <b>5</b> | <b>Analyse des métriques</b>                                                  | <b>20</b> |
| 5.1      | Latence moyenne des requêtes en fonction de la charge                         | 21        |
| 5.2      | Débit global du système en fonction de la charge                              | 21        |
| 5.3      | Efficacité du réseau : nombre moyen de sauts en fonction du nombre de nœuds   | 22        |
| 5.4      | Coût du rebalancing en fonction du nombre de nœuds                            | 23        |
| 5.5      | Distribution des clés avant et après crash                                    | 23        |
| 5.6      | Temps de convergence de la topologie pour DHT naïve                           | 24        |
| 5.7      | Synthèse comparative                                                          | 24        |
| <b>6</b> | <b>Conclusion</b>                                                             | <b>25</b> |
| 6.1      | Usages réels et intérêts pratiques des trois implémentations                  | 25        |
| 6.1.1    | Version centralisée                                                           | 25        |
| 6.1.2    | Version DHT non structurée                                                    | 25        |
| 6.1.3    | Version DHT structurée inspirée de Chord                                      | 25        |
| 6.2      | Perspectives et optimisations futures                                         | 25        |
| 6.2.1    | Version Centralisée                                                           | 25        |
| 6.2.2    | Version DHT non structurée                                                    | 26        |
| 6.2.3    | Version DHT Chord                                                             | 26        |
| 6.3      | Ouverture : vers des architectures hybrides                                   | 26        |
|          | <b>Références</b>                                                             | <b>27</b> |

# 1 Trois générations d'architectures de stockage distribué

La croissance du volume de données a rendu les architectures centralisées insuffisantes. Les systèmes distribués sont devenus la norme, mais posent un défi central : garantir un *lookup* efficace, la *consistency* et la *fault tolerance* dans un environnement de nœuds peu fiables.

Dans ce projet Master, nous explorons l'évolution des architectures de stockage distribué à travers trois générations successives, chacune répondant aux limites de la précédente :

**1. L'architecture centralisée (inspirée de Napster)** Un serveur central coordonne l'ensemble du réseau, mais cette approche reste vulnérable à un point de défaillance unique (*single point of failure*).

**2. L'architecture pair-à-pair non structurée (inspirée de Gnutella)** Elle supprime la dépendance au serveur central au profit d'un modèle P2P (architecture *symmetric*) où chaque nœud est à la fois client et serveur, au prix d'un *overhead* de communication élevé et de résultats non garantis.

**3. L'architecture DHT structurée (inspirée de Chord)** Elle structure l'*overlay* pour garantir un routage déterministe en  $O(\log N)$ , offrant le meilleur compromis entre scalabilité, résilience au *churn* et coût de maintenance.

## 1.1 Étude des approches P2P structurée: Les DHTs

Cette section résume les principes essentiels des *Distributed Hash Tables* (DHTs) : localisation des données, routage, gestion des nœuds et interfaces applicatives.

**Référence** : WEHRLE, GÖTZ et RIECHE [1].

### 1.1.1 Introduction au problème du Lookup

Le problème central est le *lookup problem* : où stocker une donnée  $D$ , puis comment la retrouver dans un réseau distribué sans coordination centrale.

"Où stocker, et comment trouver un certain élément de donnée  $D$  dans un système distribué sans aucun contrôle ou coordination centralisés ?"

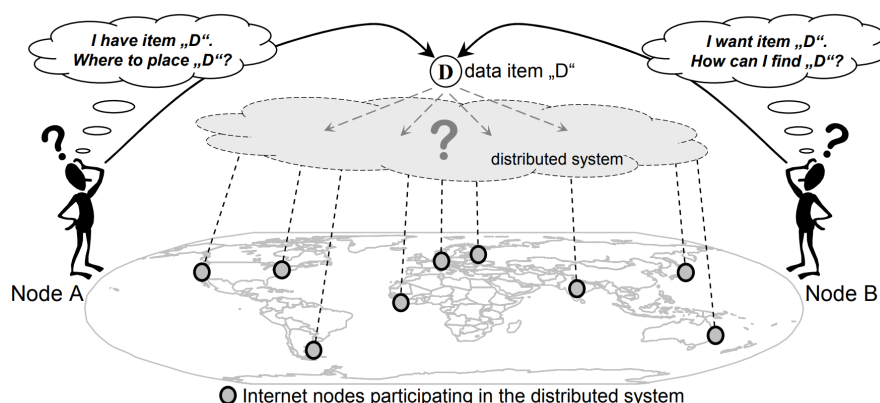


FIGURE 1 – **Le problème du lookup** : le nœud A stocke une donnée  $D$  et le nœud B doit la retrouver sans connaître son emplacement. [Source:schéma pris des notes de cours [1]]

### 1.1.2 Comparaison des stratégies de recherche

Trois familles de solutions sont comparées.

#### 1. Central Server

Approche de **première génération** (ex. Napster). Un serveur central indexe les contenus. Recherche rapide en  $O(1)$ , mais dépendance forte à un *single point of failure*.

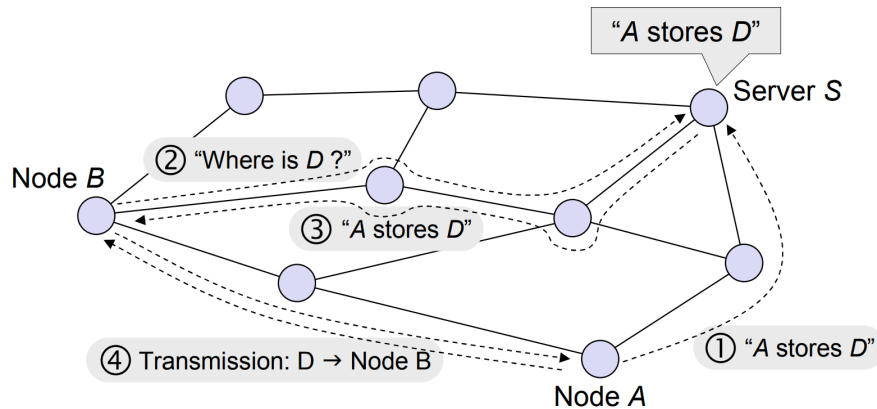


FIGURE 2 – **Central Server** : publication, requête au serveur central, puis téléchargement direct entre pairs. [Source:schéma pris des notes de cours [1]]

#### 2. Flooding Search

Approche de **deuxième génération** (ex. Gnutella). La requête est propagée de voisin en voisin via un *reactive flooding* jusqu'au *Time To Live* (TTL). L'approche évite le serveur central, mais génère un fort *communication overhead* et des résultats non garantis.

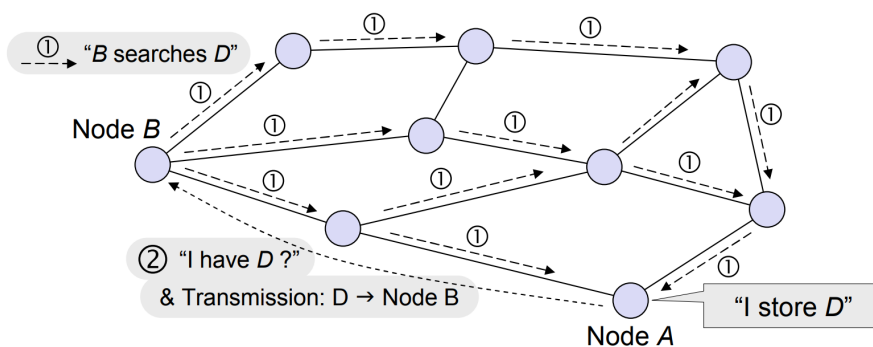


FIGURE 3 – **Flooding Search** : diffusion récursive de requêtes, puis retour des nœuds ayant la donnée. [Source:schéma pris des notes de cours [1]]

### 3. Distributed Hash Tables (DHTs)

Les DHTs (*structured Peer-to-Peer systems*) structurent l'*overlay* pour obtenir un compromis robuste :  $O(\log N)$  entrées de routage par nœud et  $O(\log N)$  hops pour localiser une donnée.

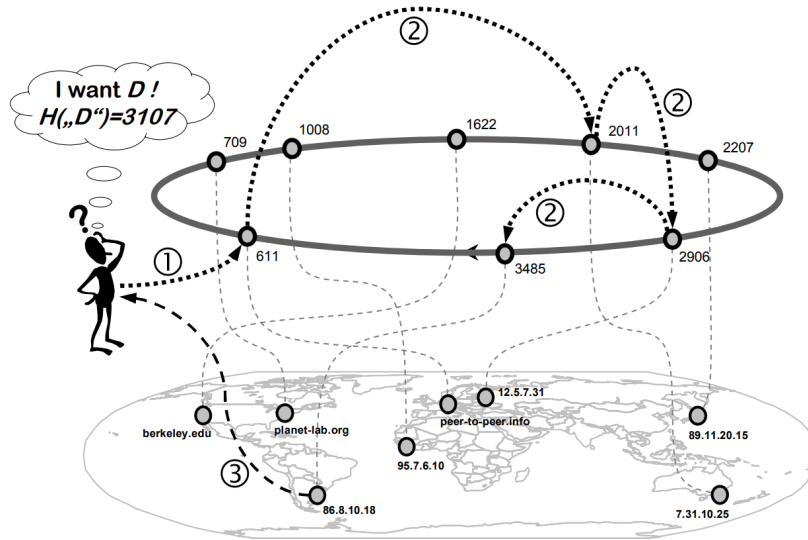


FIGURE 4 – **Distributed Hash Table** : routage structuré et efficace vers le nœud responsable. [Source:schéma pris des notes de cours [1]]

Les Figures 2, 3, 4 et 5 montrent que les DHTs offrent le meilleur compromis entre coût de recherche et coût de stockage.

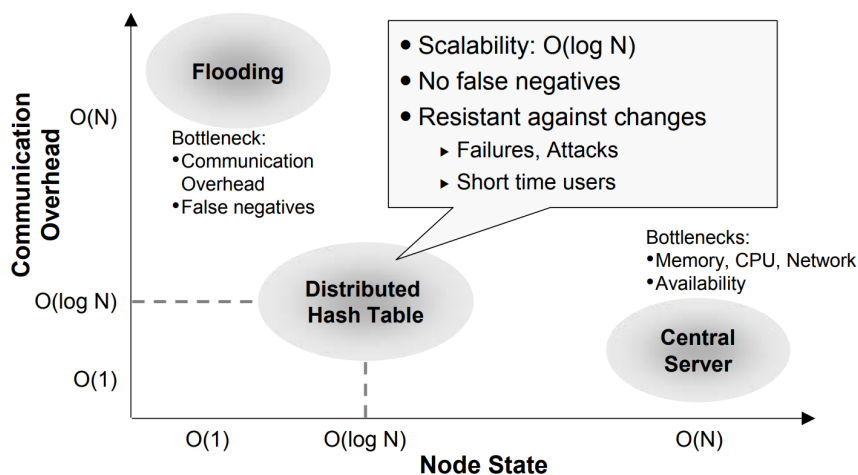


FIGURE 5 – Comparaison de la complexité en termes d'effort de recherche (axe y) et de coût de stockage par nœud (axe x). Les goulots d'étranglement (*bottlenecks*) ainsi que les caractéristiques spécifiques de chaque approche sont indiqués. [Source:schéma pris des notes de cours [1]]

### 1.1.3 Fondamentaux des Distributed Hash Tables

Une DHT associe chaque clé à un nœud responsable via un espace d'adressage logique et un routage déterministe.

#### Gestion distribuée des données

Chaque nœud gère une plage de clés et conserve une *partial view* du système. Les messages sont relayés de nœud en nœud jusqu'à la destination.

#### Adressage dans les Distributed Hash Tables

Nœuds et données partagent le même *address space* logique (souvent très grand, p. ex. 0 à  $2^{160} - 1$ ). Les IDs sont produits par une *collision-resistant hash function*.

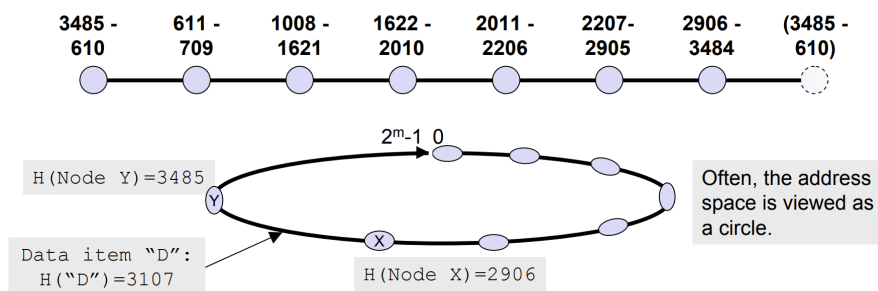


FIGURE 6 – Espace d'adressage linéaire partitionné entre paires. [Source:schéma pris des notes de cours [1]]

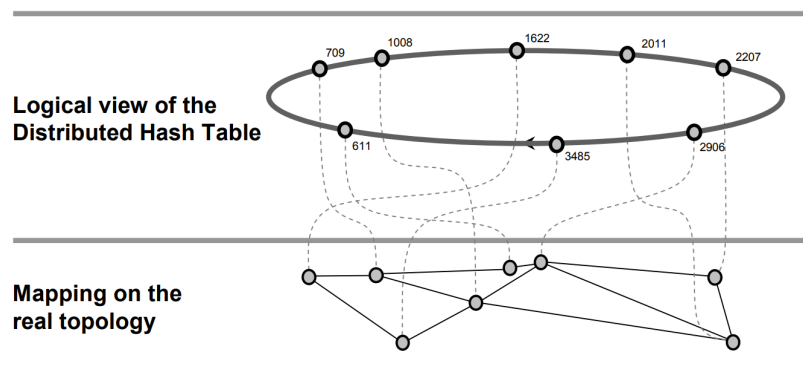


FIGURE 7 – Vue logique de la DHT et vue du réseau sous-jacent. [Source:schéma pris des notes de cours [1]]

#### Routage

Chaque nœud transmet vers un voisin plus proche de l'ID cible, ce qui réduit progressivement la distance jusqu'au nœud responsable en un *small number of hops*.

#### Méthodes de stockage

**Direct Storage** : la donnée est copiée sur le nœud responsable (meilleure *availability*, coût plus élevé).

**Indirect Storage** : la DHT stocke un *pointer* vers la donnée (coût réduit, dépendance au nœud source).

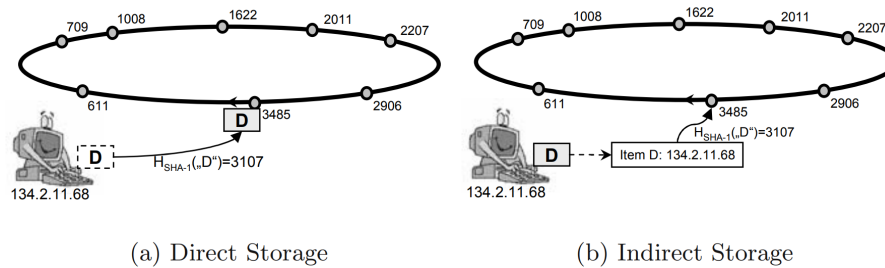


FIGURE 8 – Comparaison entre stockage direct et stockage indirect. [Source:schéma pris des notes de cours [1]]

### 1.1.4 Mécanismes des DHTs

La robustesse dépend de la gestion du *churn*, c'est-à-dire le taux de *joins* et *leaves* (arrivées/départs) ainsi que les pannes fréquentes.

#### Arrivée de nœuds

L'entrée d'un nœud suit quatre étapes courtes :

1. **Bootstrap** : le nouveau pair contacte un nœud existant qui sert d'*entry point*.
2. **Address Assignment** : il reçoit sa partition dans le *logical address space*.
3. **Routing Update** : les tables de routage sont ajustées pour intégrer le nouveau pair.
4. **Data Transfer** : les paires (*key*, *value*) devenues sa responsabilité lui sont transférées.

#### Défaillance de nœuds

Deux approches coexistent : *reactive recovery* (détection pendant le routage, puis route alternative) et *proactive recovery* (*probes* périodiques, remplacement préventif des entrées invalides). La *replication* limite la perte de données applicatives.

#### Départ de nœuds

Un départ volontaire (*graceful leave*) facilite la migration immédiate des données vers le *successor* et accélère la convergence des tables de routage. Il améliore aussi la *routing efficiency* et stabilise les mécanismes de *replication* et de *load-balancing*.

### 1.1.5 Interfaces des DHTs

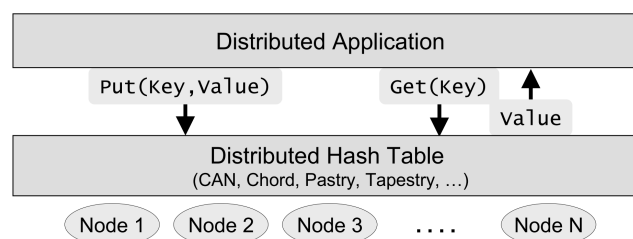


FIGURE 9 – Interface d'une Distributed Hash Table, typiquement put/get. [Source:schéma pris des notes de cours [1]]

## Interface de routage

Interface *stateless* orientée messages : *send* livre vers un ID destination, *receive* remet les messages à l'application (*delivery semantics*). Cette *stateless interface* se concentre sur la livraison de paquets basés sur un ID logique plutôt que sur une adresse IP physique.

## Interface de stockage

Interface clé-valeur : *put* écrit la donnée, *get* la récupère. La couche gère *routing failures*, *replication* et *admission control*. Cette couche ajoute de la complexité car elle gère la persistance et la fiabilité des données, comme une table de *hash* distribuée.

## Interface Client

Des hôtes peuvent consommer le service DHT sans participer au routage de l'*overlay* (mode *client*). Ce mode est utile quand on veut séparer usage applicatif et participation au routage (fiabilité, politiques d'*access control*).

### 1.1.6 Récapitulatif

Les DHTs offrent une excellente *scalability* sans point central, mais imposent des compromis entre efficacité de routage, coût de maintenance et résilience au *churn*.

## Défis de conception

- **Routing Efficiency** : qualité du routage selon la structure des tables et la topologie.
- **Management Overhead** : coût de maintenance en continu.
- **Churn** : impact des arrivées/départs simultanés sur la stabilité.

## Limitations fondamentales

- **Untrusted Environments** : résistance difficile face aux nœuds malveillants et aux *Byzantine faults*.
- **Query Constraints** : *fuzzy searches* et requêtes textuelles complexes peu adaptées au modèle par identifiants.

## 1.2 Conclusion et fil conducteur du projet

Ces trois générations forment une progression naturelle, chacune répondant aux limitations précises de la précédente. Pour en saisir pleinement les enjeux, nous allons les étudier chacune en détail.

**Notre progression dans le document** Nous commencerons par l'architecture centralisée inspirée de Napster, dont la simplicité sert de point de départ et de référence.

Nous verrons ensuite comment l'architecture non structurée inspirée de Gnutella tente de s'affranchir du point central de défaillance, et quelles nouvelles limitations elle introduit.

Enfin, nous présenterons l'architecture DHT structurée inspirée de Chord, qui résout les problèmes fondamentaux de placement et de routage identifiés dans les deux générations précédentes.

**Ouverture vers l'évaluation** Pour clore cette étude, une section dédiée aux mesures de performances mettra en regard les trois architectures, afin de mettre en lumière leurs différences concrètes en terme de scalabilité et rapidité.

## 2 Première génération : architecture centralisée inspirée de Napster

### 2.1 Vue d'ensemble de l'architecture

Cette première implémentation adopte une architecture centralisée inspirée de *Napster*. L'objectif est de proposer un système de stockage clé-valeur simple à raisonner, dans lequel un composant central conserve une vue globale des nœuds actifs et décide du placement des données. Cette approche sert de point de départ pour comparer ensuite ses limites avec des architectures plus distribuées.

Cette première architecture repose sur deux types de composants :

- Un coordinator : composant unique jouant le rôle de point d'entrée pour toutes les requêtes clientes. Il calcule le nœud responsable d'une clé et relaie les opérations vers celui-ci.
- Des nœuds de stockage : Connus et accessibles uniquement par le coordinator, sert à stocker les données des clients.

La distribution des données repose sur une fonction de hachage modulaire : pour une clé donnée, le nœud responsable est identifié par  $\text{hash}(\text{key}) \% N$ , où  $N$  désigne le nombre courant de nœuds actifs. Mathématiquement, si l'on note  $(k)$  la clé et  $(N)$  le nombre de nœuds actifs, alors le nœud responsable est donné par :

$$\text{node}(k) = h(k) \bmod N$$

L'ensemble des communications entre les composants s'effectue via des sockets TCP. Chaque message est sérialisé en Java et typé, encapsulant toutes les informations nécessaires au traitement de la requête : clé, valeur, source et compteur de sauts. Côté nœud de stockage, chaque connexion entrante est traitée dans un thread dédié créé dynamiquement, ce qui permet au nœud de gérer plusieurs requêtes simultanées sans blocage.

### 2.2 Fonctionnement des opérations fondamentales

#### 2.2.1 Opération PUT

La Figure 10 illustre le déroulement d'une requête d'écriture. Lorsque le client 1 souhaite insérer la paire  $(40, 25)$ , il envoie un message `CLIENT_PUT` au coordinator. Ce dernier applique la fonction de hachage modulaire  $\text{hash}(40) \% 5 = 0$  ce qui désigne le nœud 8001 comme unique responsable de cette clé. Le coordinator transmet alors un message `NODE_PUT` directement à ce nœud. Le nœud 8001 stocke la paire localement et répond par un `ACK`. Le coordinator retransmet enfin une `CLIENT_RESPONSE(OK)` au client pour confirmer le succès de l'opération. On observe que les nœuds 8002 à 8005, représentés en tirets, ne sont à aucun moment sollicités.

À noter que si aucun nœud actif n'est disponible au moment de la requête, le coordinator retourne directement une erreur au client.

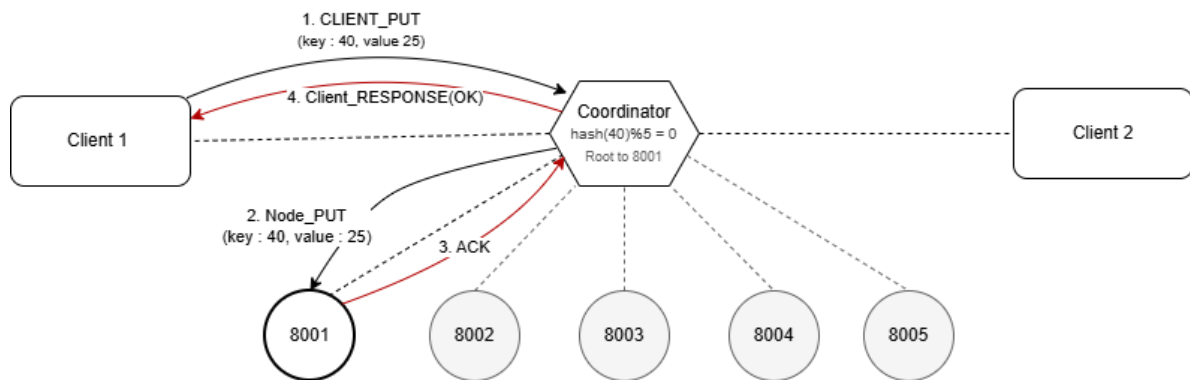


FIGURE 10 – Déroulement d'une opération PUT dans l'architecture centralisée.

Nombre total de message échangé au sein du réseau = 4 (client → coordinator → node → coordinator → client)

## 2.2.2 Opération GET

La Figure 11 illustre le déroulement d'une requête de lecture. Le client 2 envoie un CLIENT\_GET portant la clé 40. Le coordinator recalcule  $\text{hash}(40) \% 5 = 0$  et identifie de nouveau le nœud 8001 comme responsable. Un NODE\_GET est transmis au nœud 8001, qui effectue une recherche dans son store local et retourne la valeur 25 via un NODE\_RESPONSE. Le coordinator retransmet enfin une CLIENT\_RESPONSE(value : 25) correspondant à la valeur associée à la clé 40.

À noter que si la clé recherchée n'existe pas, le coordinator envoie tout de même un NODE\_GET au nœud responsable. Ce dernier effectue la recherche dans son store local, ne trouve pas la clé et retourne un NODE\_RESPONSE indiquant que la clé est absente. Le coordinator retransmet alors cette information au client sous la forme d'une CLIENT\_RESPONSE signalant que la clé n'existe pas.

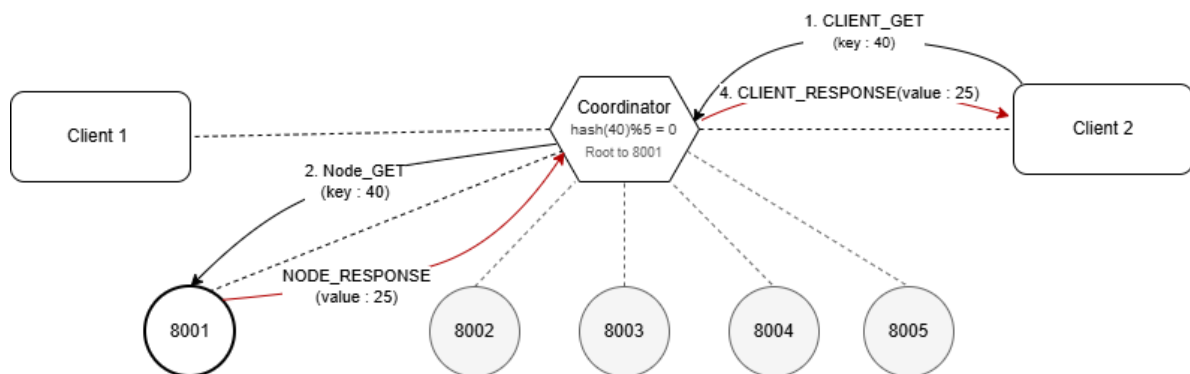


FIGURE 11 – Déroulement d'une opération GET dans l'architecture centralisée.

Nombre total de Message = 4

## 2.2.3 Opération Delete

La suppression d'une clé suit exactement la même logique que le GET. Le client envoie un CLIENT\_DELETE avec la clé de la donnée que le client souhaite supprimer, le coordinator calcule le nœud responsable et lui transmet un NODE\_DELETE. Si la clé existe, le nœud supprime la clé de son store local et indique dans sa réponse au coordinator que la clé a bien été supprimé. Si la clé n'existe pas, le nœud envoie au coordinator que la clé n'existe pas. Le coordinator transmet cette réponse reçue au client avec CLIENT\_RESPONSE.

Nombre total de Message = 4

## 2.3 Gestion dynamique des nœuds

La principale problématique avec un système distribué basé sur un calcul hachage modulaire est la valeur  $N$  désignant le nombre de nœuds présent dans le réseau. En effet, si un nœud rejoint ou quitte le réseau, cela modifie le résultat du calcul hachage modulaire et donc provoque un échange massif de flux de donnée. En effet, étant donné que le résultat a changé les données stockées doivent migrer vers le bon nœud auquel ils appartiennent après la modification du calcul hachage modulaire.

### 2.3.1 Intégration d'un nouveau nœud (JOIN)

Lorsqu'un nouveau nœud démarre, il s'enregistre auprès du coordinator en envoyant un message `REGISTER_NODE`. Le coordinator l'ajoute à sa liste de nœuds actifs et déclenche immédiatement un rebalance. En effet, l'ajout d'un nœud fait passer le modulo de  $N$  à  $N+1$ , ce qui modifie le nœud responsable des clés existantes. Le rebalance ne s'agit pas uniquement d'envoyer les données au nouveau venu. En effet, les emplacements de toutes les données sont recalculés et certaines peuvent migrer entre deux nœuds déjà présents sans impliquer le nouveau nœud.

Le rebalance commence par un inventaire complet : le coordinator envoie un `DUMP_REQUEST` à chaque nœud actif, qui lui retourne la liste de ses clés. À partir de cet inventaire, le coordinator recalcule pour chaque clé le nœud qui devrait en être responsable avec le nouveau  $N+1$  et construit un plan de migration précis indiquant, pour chaque nœud source, quelles clés doivent être envoyées à quel nœud destination.

Pour chaque entrée de ce plan, le coordinator envoie un message `TRANSFER_KEYS` au nœud source, en lui communiquant la destination et la liste des clés concernées. Le nœud source collecte les valeurs correspondantes dans son store local, les regroupe en un seul message `NODE_BATCH_PUT` et l'envoie directement au nœud destination, sans que les valeurs ne transitent par le coordinator. Le nœud destination stocke le batch et répond `ACK`. Le nœud source supprime alors les clés transférées de son propre store.

Pendant toute la durée de ces transferts, les requêtes clients sont temporairement bloquées. Lorsque le plan de migration contient des déplacements à effectuer, le coordinator envoie au préalable un message `REBALANCE` à tous les nœuds, qui positionnent un flag interne `rebalancing = true`. Toute requête `NODE_PUT`, `NODE_GET` ou `NODE_DELETE` reçue pendant cet état retourne immédiatement un message `REBALANCING` au coordinator, qui le propage au client. Ce blocage garantit qu'aucun client ne tente de lire une clé en cours de déplacement, ce qui éviterait une réponse incorrecte de type `NOT_FOUND` sur une donnée qui existe. À la fin du rebalance, le coordinator diffuse un `REBALANCE_DONE` à tous les nœuds. À la réception de ce message, les nœuds vont positionner leur flag interne `rebalancing = false` pour débloquer les requêtes clients.

### 2.3.2 Retrait d'un nœud (LEAVE)

Lorsqu'un nœud quitte volontairement le réseau, son `ShutdownHook` envoie un message `UNREGISTER_NODE` au coordinator. La première action du coordinator, avant de retirer ce nœud de sa liste, est d'envoyer un `DUMP_REQUEST` pour récupérer l'intégralité des données qu'il détient. Cette étape est critique : si le dump est effectué après la déconnexion, les données sont définitivement perdues. Une fois les données orphelines récupérées, le coordinator retire le nœud de sa liste et déclenche le rebalance.

Le traitement des données orphelines diffère légèrement du JOIN. Ces données, récupérées par dump avant la déconnexion, sont injectées directement par le coordinator via des messages `NODE_PUT_INTERNAL` vers les nœuds cibles car le nœud source est désormais hors ligne et ne peut plus émettre de `TRANSFER_KEYS`. En revanche, pour les clés déjà présentes sur les nœuds restants qui se retrouvent mal placées suite au changement de modulo de  $N$  à  $N-1$ , le mécanisme est identique au JOIN : le coordinator envoie des `TRANSFER_KEYS` aux nœuds sources, qui transfèrent directement leurs données aux nœuds destinations via `NODE_BATCH_PUT`. Le blocage des clients via le flag `rebalancing` est activé

pour les mêmes raisons que lors d'un JOIN.

### 2.3.3 Détection des pannes par heartbeat

Chaque nœud envoie périodiquement un message HEARTBEAT au coordinator toutes les cinq secondes. Un thread de surveillance vérifie régulièrement ces timestamps : si un nœud n'a pas donné signe de vie depuis plus de quinze secondes, il est considéré comme mort et retiré de la liste des nœuds actifs. Le coordinator déclenche alors un rebalance pour redistribuer les clés selon le nouveau N.

A noter que lors du retrait d'un nœud à cause d'une panne, celui-ci quitte le réseau de manière involontaire. Le nœud étant mort, il est impossible d'effectuer un dump préalable de ses données. Les clés qu'il détenait sont donc définitivement perdues. C'est une des limitations de cette implémentation, qui ne dispose pas de mécanisme de réplication.

## 2.4 Avantages et limites de l'approche centralisée

L'architecture centralisée présente plusieurs avantages notables. Sa simplicité conceptuelle facilite le débogage et la compréhension du système : un seul composant détient la vue globale du cluster, ce qui rend le routage déterministe et prévisible. La cohérence des données est naturellement garantie puisque toutes les décisions de placement transitent par un unique point de calcul. Le rebalance node-to-node améliore par ailleurs la redistribution en évitant que le coordinator ne transporte les données.

Cependant, cette architecture présente des limites structurelles importantes. Le coordinator constitue un point de défaillance unique. En effet, si le coordinator tombe en panne, l'ensemble du système devient inaccessible. L'absence de réplication représente également une faiblesse majeure : en cas de panne d'un nœud, les données qu'il détenait sont irrémédiablement perdues. Enfin, le hachage modulaire classique entraîne une redistribution massive des données à chaque changement de topologie, comportement qui sera directement comparé à l'approche par hachage cohérent implémentée dans les versions DHT suivantes.

## 3 Deuxième génération : architecture non structurée inspirée de Gnutella

### 3.1 Vue d'ensemble de l'architecture

Cette deuxième implémentation adopte une architecture pair-à-pair non structurée, inspirée dans son principe de *Gnutella 0.4*. L'objectif est de supprimer la dépendance au coordinateur central de la première génération, en confiant à chaque nœud un rôle simultané de stockage, de découverte des pairs et de routage des requêtes. Cette approche sert de transition vers des architectures plus structurées, en mettant en évidence les limites d'un réseau entièrement décentralisé mais sans espace d'identifiants stable.

Le système suit un modèle *pure P2P*. Il n'existe aucune séparation stricte entre clients et serveurs : chaque processus est un *servent* (*server + client*) reposant sur trois composants principaux :

- Un registre local des pairs : contient les voisins connus et sert à établir de nouvelles connexions.
- Un stockage local : conserve les paires (*key*, *value*) dont le nœud dispose effectivement.
- Une couche de routage : retransmet les requêtes GET, filtre les doublons et maintient temporairement les informations nécessaires au retour des réponses.

Le placement des données repose sur la même fonction de hachage modulaire que la première

génération. Pour une clé donnée, le propriétaire logique est déterminé localement par  $\text{hash}(\text{key}) \% N$ , où  $N$  désigne le nombre de nœuds connus par le pair qui émet la requête :

$$\text{owner}_N(h) = h \bmod N$$

Contrairement à la première génération, cette valeur de  $N$  n'est plus une vérité globale maintenue par un coordinateur : elle dépend de la vue locale de chaque pair, ce qui introduit des risques d'incohérence de placement dès que les vues divergent ou que  $N$  varie sous l'effet du *churn*.

Les communications s'effectuent également via des sockets TCP. Chaque message est sérialisé en Java et typé, encapsulant la clé, la valeur éventuelle, l'origine de la requête, un identifiant unique de message et un compteur de sauts. Côté réception, chaque connexion entrante est traitée dans un thread dédié créé dynamiquement, ce qui permet à chaque nœud de gérer plusieurs requêtes simultanées sans blocage.

## 3.2 Principe de fonctionnement

Dans une DHT structurée, tous les nœuds partagent un espace logique stable et une règle précise pour déterminer le responsable d'une clé. Dans cette version non structurée, le placement reste plus naïf : un pair peut calculer un propriétaire logique à partir d'une fonction de hachage et du nombre de pairs connus. Cette stratégie est simple et entièrement décentralisée, mais elle fait apparaître plusieurs difficultés : les vues locales peuvent diverger, le placement dépend du nombre de nœuds, et la recherche ne dispose pas encore d'un chemin déterministe vers la donnée.

Le fonctionnement général combine donc plusieurs mécanismes complémentaires. Le PUT utilise d'abord une règle de placement locale, suffisante lorsque la vue du réseau est cohérente. Le GET s'appuie ensuite sur un flooding contrôlé pour retrouver une donnée même si le propriétaire logique calculé localement est incorrect. La gestion du JOIN vise à propager rapidement l'adresse des nouveaux pairs afin de densifier la topologie, tandis que le *reverse path caching* réutilise le chemin de retour des réponses pour créer des caches intermédiaires. Ces mécanismes ne transforment pas le système en DHT structurée, mais ils compensent partiellement les limites d'un réseau non structuré.

Le premier problème concerne la cohérence de la vue réseau. Dans une topologie clairsemée, chaque pair ne connaît qu'un sous-ensemble du réseau ; deux pairs peuvent donc calculer des propriétaires différents pour une même clé. La Figure 12 illustre cette limite : sans espace d'adressage partagé et stable, la cohérence du placement dépend de la cohérence de la vue réseau.

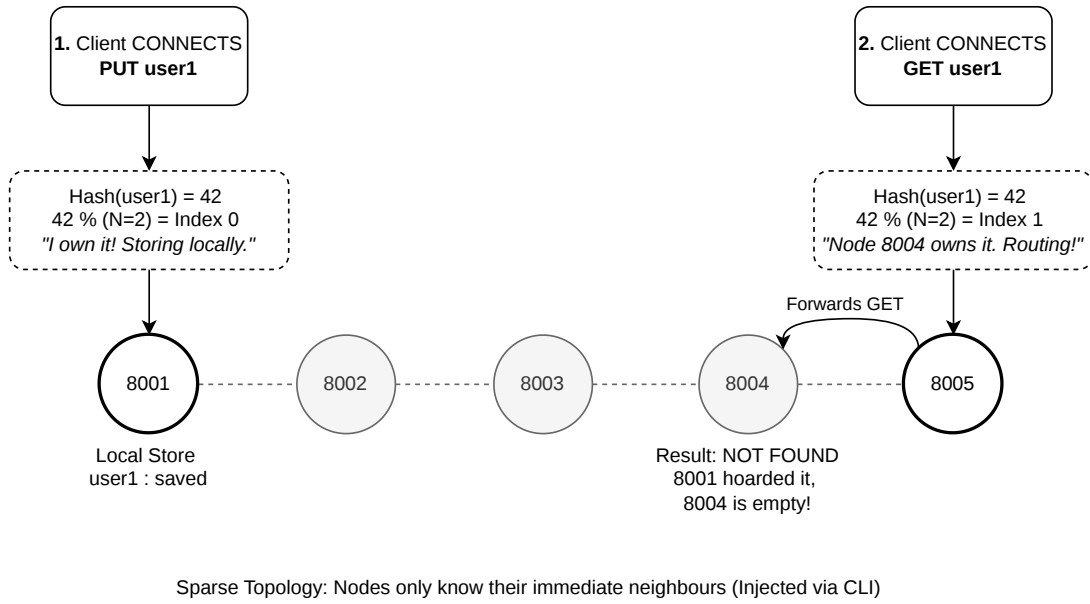


FIGURE 12 – Problème de vue globale dans une topologie clairsemée.

Le nœud 8001 stocke localement `user1` après avoir calculé  $\text{Hash}(\text{user1}) = 42$  avec sa propre vision du système. Plus tard, le nœud 8005 effectue un GET pour la même clé, mais avec une vue réseau différente il conclut que le propriétaire devrait être 8004 et relaie donc la requête dans une mauvaise direction. L'exemple montre qu'en l'absence d'un espace d'adressage partagé et stable, deux nœuds peuvent prendre des décisions incompatibles pour une même clé.

Une réponse naturelle consiste à densifier la topologie pour que les vues locales convergent. Si l'on note  $V$  l'ensemble des nœuds actifs et  $E$  l'ensemble des liens de voisinage, un mesh complet correspond au graphe non orienté

$$G = (V, E), \quad E = \{\{u, v\} \mid u, v \in V, u \neq v\}.$$

Pour  $|V| = N$ , le nombre total d'arêtes vaut alors

$$|E| = \binom{N}{2} = \frac{N(N-1)}{2},$$

et chaque nœud a un degré

$$\deg(v) = N - 1.$$

Cette topologie améliore fortement la cohérence de la vue réseau en régime stable, mais elle introduit un coût de maintenance quadratique.

### 3.3 Opération PUT

L'opération PUT consiste à associer une clé à une valeur et à placer cette donnée sur un nœud jugé responsable. Dans le schéma naïf, le responsable peut être déterminé par une règle de type

$$\text{owner}_N(h) = h \bmod N,$$

où  $h$  est le hachage de la clé et  $N$  le nombre de nœuds connus localement. Cette règle a l'avantage d'être facile à calculer et ne nécessite pas de serveur central.

Sa limite principale est sa dépendance directe à  $N$ . Si un nœud rejoint ou quitte le réseau, la valeur de  $N$  change et le même hachage peut désigner un autre propriétaire logique :

$$\text{owner}_{N-1}(h) = h \bmod (N - 1).$$

Par exemple, avec  $h = 10$  et  $N = 4$ , on obtient

$$10 \bmod 4 = 2, \quad 10 \bmod 3 = 1.$$

Le propriétaire logique change donc alors même que la donnée n'a pas physiquement bougé.

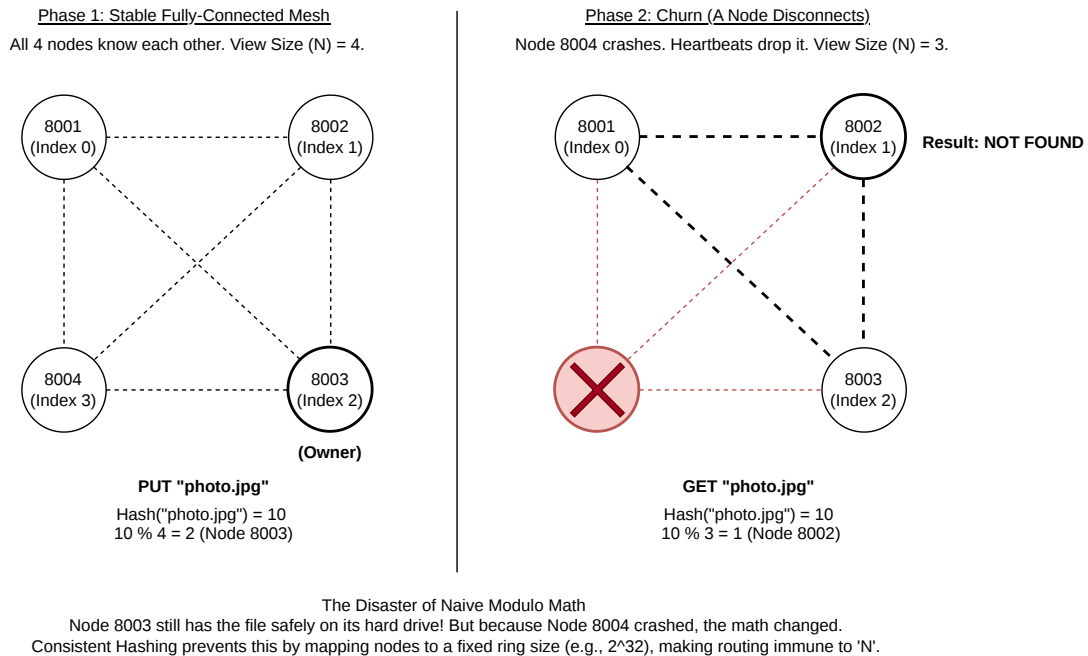


FIGURE 13 – Impact du *churn* sur un placement basé sur  $\text{hash}(\text{key}) \% N$ .

À gauche, avec quatre nœuds connectés en mesh, la clé `photo.jpg` est associée au nœud 8003, qui la stocke localement. À droite, après la panne du nœud 8004, la taille de la vue passe de 4 à 3 ; le même calcul désigne alors 8002 comme propriétaire logique. Le mesh améliore donc la cohérence de la vue tant que le système est stable, mais il ne protège pas contre l'instabilité du placement lorsque  $N$  varie.

Le PUT est donc correct lorsque la vue des pairs est cohérente et stable, mais il n'offre pas de garantie forte sous *churn*. Contrairement au hachage cohérent utilisé dans les DHT structurées, le modulo naïf peut déplacer logiquement une grande partie de l'espace de clés après une simple variation du nombre de participants.

### 3.4 Opération GET et routage par flooding

L'opération GET doit retrouver une clé même lorsque le propriétaire logique calculé localement est faux ou incomplet. Pour cela, l'architecture utilise une inondation contrôlée : si un nœud ne possède pas la donnée demandée, il retransmet la requête à ses voisins connus.

- **Diffusion.** Une requête est rediffusée de pair en pair jusqu'à rencontrer un nœud possédant la clé.
- **TTL.** Un plafond `MAX_HOPS = 10` limite le nombre de sauts pour éviter les boucles infinies.
- **Élimination des doublons.** Le registre local mémorise les identifiants déjà vus, par exemple sous la forme `Type|Origin|Seq`, afin d'ignorer les messages dupliqués.

Dans un graphe de degré moyen  $d$ , une inondation de profondeur maximale  $T$  génère, dans le pire

cas sans filtrage efficace, un nombre de transmissions borné par

$$1 + d + d^2 + \dots + d^T = \sum_{i=0}^T d^i.$$

Si  $d > 1$ , cette somme vaut

$$\sum_{i=0}^T d^i = \frac{d^{T+1} - 1}{d - 1}.$$

Le coût peut donc croître très vite avec la profondeur de recherche. Dans l'implémentation, le cache de messages déjà vus limite cette explosion : une requête touche essentiellement les arêtes accessibles un nombre borné de fois. Dans un mesh complet, où  $|E| = N(N - 1)/2$ , une recherche qui explore tout le réseau reste néanmoins en  $\Theta(N^2)$  transmissions dans le pire cas.

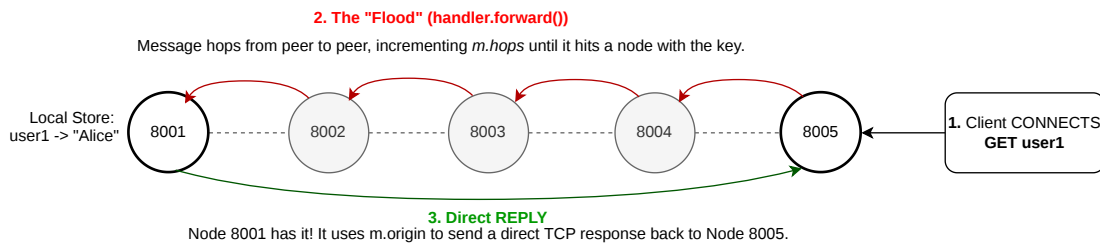


FIGURE 14 – Recherche par flooding contrôlé.

Le nœud demandeur diffuse un GET à ses voisins ; chaque pair relaie la requête tant que le TTL n'est pas épuisé et que le message n'a pas déjà été vu. Lorsqu'un nœud possède la donnée, il peut produire une réponse REPLY.

Le flooding joue ainsi le rôle d'un mécanisme de rattrapage. Il compense l'absence de routage structuré en augmentant la probabilité d'atteindre le détenteur réel de la donnée, mais cette robustesse s'obtient au prix d'un trafic réseau important.

## 3.5 Gestion dynamique des nœuds

La gestion dynamique des nœuds est centrale dans cette architecture, car la cohérence de la vue réseau conditionne directement le placement et la recherche. Deux mécanismes doivent être distingués : la découverte progressive par messages PING/PONG, et la diffusion explicite d'un nouveau pair lors du JOIN.

### 3.5.1 Intégration d'un nouveau nœud (JOIN)

Une approche minimale consiste à faire rejoindre un nouveau nœud via un pair de bootstrap. Le nouveau participant connaît l'adresse d'un seul nœud existant, s'y connecte, puis échange des messages PING/PONG. Avec un mécanisme de *pong caching*, les réponses PONG ne contiennent pas seulement l'adresse du répondant, mais aussi quelques pairs déjà connus localement. Cette stratégie permet à une topologie initialement clairsemée de converger progressivement vers un réseau plus connecté.

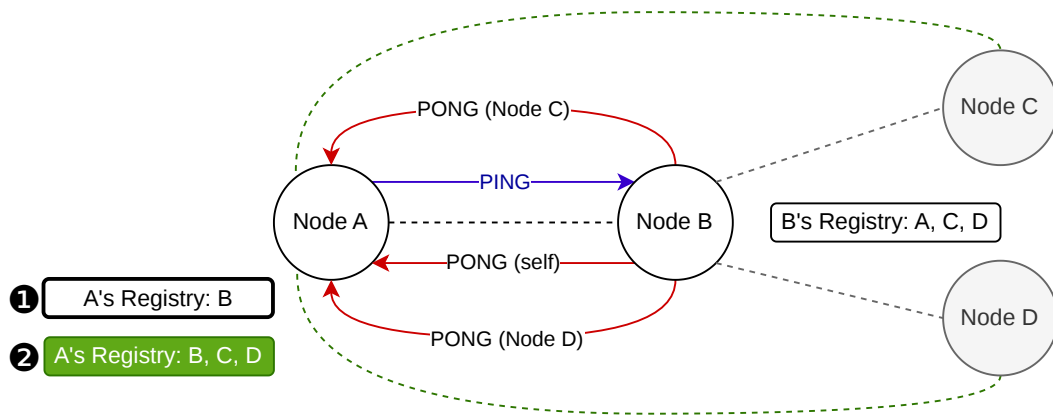


FIGURE 15 – Principe du *pong caching*.

En plus d'annoncer sa propre présence, un nœud répondant à un PING peut transmettre quelques adresses supplémentaires issues de son registre local. Par diffusion répétée de ces informations, les pairs convergent plus vite vers une vue commune des participants actifs.

Dans l'implémentation finale, le JOIN va plus loin que cette découverte progressive. Lorsqu'un nœud rejoint le réseau via un pair déjà connu, son adresse est propagée dans le réseau. Les pairs qui reçoivent cette information peuvent l'ajouter à leur registre et établir la connexion correspondante. Ce choix court-circuite en partie le rôle du heartbeat comme mécanisme principal de découverte : le réseau devient très rapidement connecté après l'arrivée d'un nouveau participant, au lieu d'attendre plusieurs cycles de PING/PONG.

Le *pong caching* reste utile pour enrichir et réparer la vue locale, mais il n'est plus le seul moteur de convergence. La connectivité provient surtout de la diffusion explicite de l'adresse du nouveau nœud pendant le JOIN. Cette décision rend les expérimentations plus stables à petite échelle, tout en rapprochant la topologie effective d'un mesh dense.

### 3.5.2 Retrait d'un nœud (LEAVE)

Lorsqu'un nœud quitte volontairement le système, les autres pairs doivent retirer son adresse de leur registre local. Dans une architecture non structurée, cette opération ne s'accompagne pas d'une réorganisation globale comparable à celle d'une DHT structurée. Les données détenues uniquement par le nœud sortant peuvent devenir indisponibles si elles n'ont pas été répliquées ou mises en cache ailleurs.

Le départ d'un pair modifie aussi la valeur effective de  $N$  pour les nœuds qui utilisent un placement par  $\text{hash}(\text{key}) \% N$ . Même si la topologie reste connectée, le propriétaire logique d'une clé peut donc changer. Le LEAVE révèle ainsi la limite principale du modèle : la connectivité peut être maintenue, mais la stabilité du placement n'est pas garantie.

### 3.5.3 Détection des pannes et stabilisation

La détection des pannes repose sur des heartbeats et sur l'échec des communications réseau. Un pair qui ne répond plus peut être supprimé du registre local, ce qui évite de continuer à lui relayer des messages. Cette stabilisation est locale : chaque nœud corrige progressivement sa propre vue du réseau.

Dans une topologie dense, cette correction locale suffit souvent à préserver la connectivité globale, car plusieurs chemins alternatifs existent entre les participants. En revanche, elle ne reconstruit pas un espace d'adressage stable et ne déclenche pas automatiquement un transfert systématique des

clés. La stabilisation concerne donc surtout la connectivité et la vivacité, pas la cohérence globale du placement.

### 3.6 Reverse path caching

La recherche par flooding peut produire beaucoup de requêtes répétées pour les mêmes données. Pour réduire ce coût dans les cas fréquents, l'architecture introduit un mécanisme de *reverse path caching*, aussi appelé mécanisme de *breadcrumbs*. Son objectif est double : permettre à une réponse *REPLY* de revenir vers le client sans que le nœud détenteur connaisse directement son adresse, et mettre en cache la donnée sur les nœuds intermédiaires traversés par la réponse.

Le principe est le suivant. Pendant la propagation d'un *GET*, chaque nœud qui relaie la requête laisse localement un *breadcrumb* : il associe un identifiant de route, par exemple *seq:key*, au voisin depuis lequel la requête est arrivée. Si un nœud finit par trouver la donnée, il émet un *REPLY* qui suit ces traces en sens inverse. À chaque saut, le *breadcrumb* est consommé puis supprimé, ce qui évite de conserver indéfiniment des états temporaires. Le nœud intermédiaire profite alors du passage de la réponse pour stocker localement la paire (*key*, *value*).

Ce mécanisme donne donc un rôle actif au chemin de retour. Si une requête part d'un nœud éloigné et atteint finalement le détenteur de la donnée, tous les relais situés sur le chemin inverse apprennent progressivement cette donnée. Lors d'une requête ultérieure pour la même clé, un cache intermédiaire plus proche peut répondre avant que le flooding n'atteigne le détenteur initial.

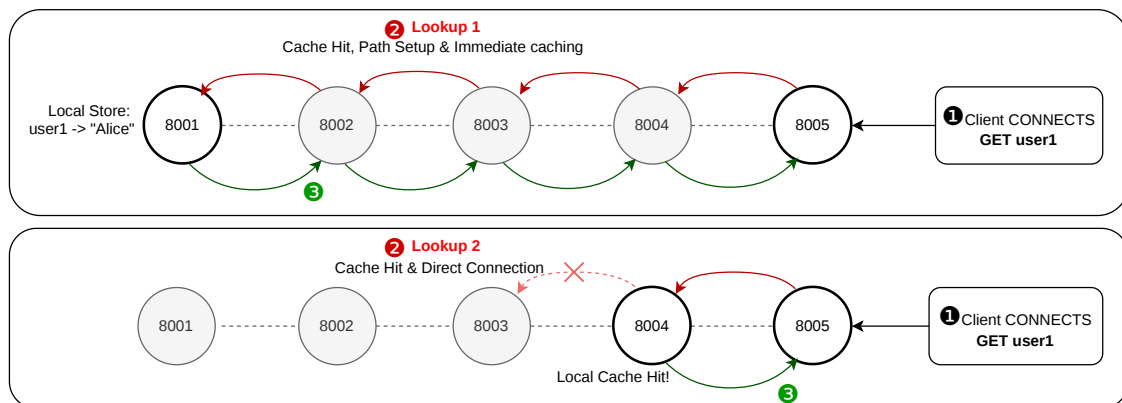


FIGURE 16 – Lecture détaillée du *reverse path caching*.

Dans la partie haute (*Lookup 1*), le chemin rouge correspond au *lookup* initial émis par le client connecté à 8005 pour obtenir *user1*. La donnée n'étant pas présente localement, il s'agit d'un *cache miss* : la requête progresse de pair en pair jusqu'au nœud 8001, qui possède la valeur dans son stockage local. Le chemin vert représente ensuite la réponse *REPLY* qui remonte en sens inverse ; à chaque saut, les nœuds intermédiaires utilisent les *breadcrumbs* laissés par le *lookup* pour relayer la réponse et mettre immédiatement la donnée en cache. Dans la partie basse (*Lookup 2*), une nouvelle requête pour la même clé repart de 8005, mais elle rencontre cette fois un *cache hit* sur un nœud intermédiaire proche, ici 8004. Le *lookup* rouge est alors plus court, et la réponse verte revient directement sans devoir remonter jusqu'au propriétaire initial 8001. Le schéma montre ainsi à la fois le premier *cache miss*, la phase d'amorçage du cache le long du chemin inverse, puis le second *cache hit* qui réduit la distance de résolution.

Ce mécanisme transforme les réponses en opportunités de réplication locale. Une première requête pour une donnée populaire peut encore parcourir une grande partie du réseau, mais les requêtes suivantes peuvent être interceptées plus tôt par un cache intermédiaire. Le routage reste non structuré, mais le coût effectif diminue pour les clés fréquemment demandées.

### 3.7 Avantages et limites de l'approche non structurée

Cette architecture présente plusieurs avantages. Elle est simple à mettre en place, ne dépend d'aucun coordinateur central et tolère assez bien les pannes locales tant que la topologie reste connectée. La diffusion explicite des nouveaux pairs pendant le `JOIN` accélère fortement la convergence vers un réseau dense. Le flooding permet de retrouver une donnée même lorsque le calcul local du propriétaire est incorrect, et le *reverse path caching* améliore les requêtes répétées.

Ses limites sont toutefois structurelles. Le placement par  $\text{hash}(\text{key}) \% N$  dépend du nombre de participants et devient instable sous *churn*. Le mesh dense améliore la cohérence des vues, mais impose une mémoire en  $\Theta(N)$  par pair et un coût global en  $\Theta(N^2)$ . Le flooding est robuste mais coûteux : dans le pire cas, une recherche sur un mesh complet peut atteindre  $\Theta(N^2)$  transmissions. Enfin, la stabilisation locale maintient la connectivité, mais ne fournit pas de garantie déterministe sur la localisation des clés.

Cette deuxième génération constitue donc une étape intermédiaire entre le serveur central et une DHT structurée. Elle montre qu'un réseau entièrement décentralisé peut fonctionner sans point de coordination unique, mais aussi que le passage à l'échelle nécessite un espace d'identifiants stable et un routage plus déterministe. Ces limites motivent directement l'introduction d'approches comme *Chord*, où la responsabilité des clés et le chemin de recherche sont définis par une structure logique explicite.

## 4 Troisième génération : DHT structurée inspirée de Chord

### 4.1 Vue d'ensemble de l'architecture

Cette troisième implémentation adopte une architecture DHT structurée inspirée de Chord. Contrairement aux générations précédentes, il n'existe ni *coordinator* central ni *flooding* : chaque nœud s'auto-organise dans un anneau logique et route de manière déterministe les requêtes vers le nœud responsable, sans connaissance globale du réseau.

Cette troisième implémentation repose sur deux concepts fondamentaux :

- Un **anneau logique** : chaque nœud occupe une position fixe sur un anneau de taille  $2^6 = 64$ , calculée par hachage de son adresse. Les clés sont distribuées sur cet anneau via *consistent hashing*.
- Une **finger table** : table de routage locale de  $M = 6$  entrées permettant à chaque nœud de router les requêtes en  $O(\log N)$  messages sans connaissance globale du réseau.

La cohérence de l'anneau est assurée par un service de stabilisation périodique qui corrige les pointeurs *successor* et *predecessor* et maintient une liste de successeurs de secours pour la tolérance aux pannes.

### 4.2 Principe de fonctionnement

Chaque nœud et chaque clé reçoivent un identifiant sur  $m$  bits (souvent via SHA-1), placé sur un anneau modulo  $2^m$ . Une clé  $k$  est stockée sur son *successor* : le premier nœud rencontré dans le sens horaire dont l'identifiant est supérieur ou à  $k$ .

Cette règle donne trois propriétés utiles :

- **Répartition** : le *consistent hashing* distribue les clés de manière quasi uniforme.
- **Scalabilité** : l'ajout/retrait d'un nœud ne déplace qu'une fraction des clés contrairement au  $\text{hash}(\text{key}) \% N$  des générations précédentes.
- **Décentralisation** : aucun serveur central n'est requis.

### 4.2.1 Routage efficace : Finger Table

Pour éviter un parcours linéaire de l'anneau, chaque nœud maintient une **finger table** de  $m$  entrées. L'entrée  $i$  pointe vers le premier nœud joignable à partir de  $n + 2^{i-1}$ .

$$\text{finger}[i].\text{start} = (n + 2^{i-1}) \bmod 2^m$$

Cette structure permet des sauts exponentiels et un coût moyen de recherche en  $O(\log N)$  messages : le routage consiste à sélectionner le finger le plus proche précédant la clé cible (`closestPrecedingFinger`), et à déléguer la requête à ce nœud, jusqu'à atteindre le *successor* de la clé.

### 4.2.2 Opération PUT

Lorsqu'un nœud reçoit une requête PUT pour une paire  $(key, value)$ , il calcule le *hash* de la clé et appelle *lookup* pour identifier le *successor* responsable. Si ce *successor* est le nœud lui-même, la paire est stockée localement. Sinon, la requête est routée vers le nœud désigné via la *finger table*. Le message transite ainsi de nœud en nœud jusqu'à atteindre le responsable, qui effectue le stockage final.

**Nombre total de messages échangés :**  $O(\log N)$

### 4.2.3 Opération GET

La lecture suit exactement la même logique que le PUT. Le nœud recevant la requête GET calcule le *hash* de la clé et identifie le *successor* responsable via *lookup*. Si c'est lui-même, il retourne la valeur depuis son *store* local, ou NOT\_FOUND si la clé est absente. Sinon, la requête est routée vers le nœud responsable, qui répond directement à l'émetteur original.

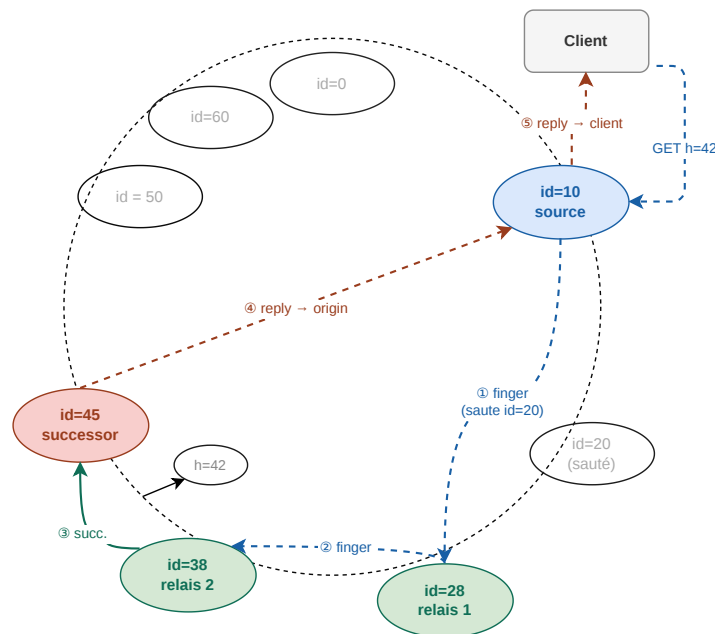


FIGURE 17 – Déroulement d'une opération GET dans l'architecture Chord.

**Nombre total de messages échangés :**  $O(\log N)$

## 4.3 Gestion dynamique des nœuds

### 4.3.1 Intégration d'un nouveau nœud (JOIN)

Lorsqu'un nouveau nœud démarre, il contacte un nœud existant passé en paramètre (`bootstrapPeer`) et appelle `remoteCallFindSuccessor` pour identifier sa position sur l'anneau. Une fois son *successor* déterminé, il lui envoie un message `REQUEST_KEYS` pour récupérer les clés dont il devient responsable. Contrairement à la première génération, ce transfert est ciblé : seules les clés appartenant à la plage (prédécesseur du *successor*, id du nouveau nœud] sont déplacées. Le service de stabilisation se charge ensuite de propager progressivement la connaissance du nouveau nœud aux autres membres de l'anneau via `notify`.

### 4.3.2 Retrait d'un nœud (LEAVE)

Lors d'un départ volontaire, le `ShutdownHook` déclenche la procédure de sortie propre. Le nœud transfère d'abord l'intégralité de son *store* local à son *successor* via `transferAllKeysTo`. Il envoie ensuite un message `LEAVE_NOTIFY` à son *successor* et à son *predecessor* pour qu'ils mettent à jour leurs pointeurs et maintiennent la cohérence de l'anneau.

### 4.3.3 Détection des pannes et stabilisation

Le service de stabilisation (`StabilizeService`) s'exécute toutes les 2 secondes. À chaque cycle, il vérifie la vivacité du *successor* via `remoteCallGetPredecessor` : si celui-ci ne répond pas, il cherche un remplaçant dans la liste de successeurs de secours, puis dans la *finger table*. Le *predecessor* est vérifié tous les 3 cycles. En cas de panne, il est remis à `null` et l'anneau se reconfigure progressivement. La *finger table* est quant à elle mise à jour entrée par entrée à chaque cycle via `fixFingers`.

## 4.4 Avantages et limites de l'approche Chord

L'architecture Chord présente plusieurs avantages majeurs par rapport aux générations précédentes. Le *consistent hashing* garantit une distribution quasi uniforme des clés et limite les transferts lors des changements de topologie : un *JOIN* ou *LEAVE* ne déplace qu'une fraction des clés, contre une redistribution massive dans la première génération. Le routage déterministe via la *finger table* offre un *lookup* en  $O(\log N)$  messages, sans *flooding* ni *coordinator* central. Enfin, la liste de successeurs de secours et la stabilisation périodique assurent une tolérance aux pannes que les générations précédentes ne permettaient pas.

Cependant, cette architecture présente également des limites. La stabilisation périodique introduit un délai de convergence : dans une période de *churn* intense, l'anneau peut temporairement être dans un état incohérent, rendant certaines clés introuvables le temps que les pointeurs se corrigent. Par ailleurs, notre implémentation utilise un espace d'adressage de taille  $2^6 = 64$ , ce qui limite le nombre de nœuds distincts pouvant coexister sur l'anneau. Enfin, comme dans la première génération, l'absence de réplication implique qu'une panne non détectée avant la stabilisation peut entraîner une perte temporaire de données.

## 5 Analyse des métriques

Cette section présente les résultats expérimentaux obtenus pour chacune des implémentations développées dans le cadre du projet. Pour chaque architecture, nous analysons quatre dimensions : la latence moyenne des requêtes sous charge croissante, le débit global du système, l'efficacité du routage mesurée en nombre de sauts, et lorsque applicable le comportement du système face aux changements de topologie.

## 5.1 Latence moyenne des requêtes en fonction de la charge

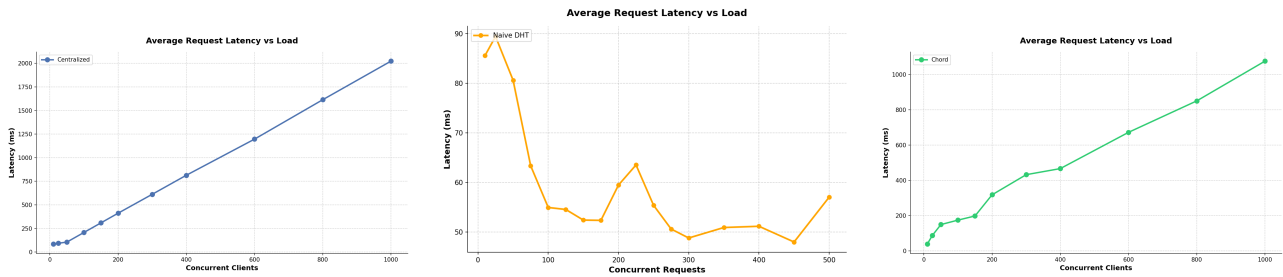


FIGURE 18 – Latence moyenne des requêtes en fonction de la charge.

La Figure 18 permet une comparaison directe du comportement en latence des trois architectures sous charge croissante.

Le centralisé est celle qui se dégrade le plus : la latence croît quasi linéairement de 80 ms à plus de 2000 ms à 1000 clients. Ce comportement s'explique par le goulot d'étranglement du coordinator central, point par lequel transitent toutes les requêtes. Sous forte charge, les files d'attente s'accumulent et le temps d'attente devient dominant.

La DHT naïve présente un comportement inverse : la latence commence haute (85 ms) puis chute rapidement vers une stabilité autour de 50 ms. C'est l'effet du *reverse path caching* — les données dites populaires se rapprochent progressivement des clients, faisant tendre le taux de cache  $h \rightarrow 1$  et la latence vers le coût d'un accès local  $L_{cache}$ , bien inférieur au coût d'un parcours réseau complet.

Chord a une latence qui croît régulièrement de 35 ms à 900 ms à 1000 clients. Le coût du routage en  $O(\log N)$  sauts s'accumule sous forte charge, mais contrairement au centralisé, la contention est distribuée sur l'ensemble des nœuds de l'anneau, ce qui atténue la dégradation. Chord subit donc un surcoût lié au routage, mais évite le point de saturation brutal du centralisé.

En résumé, la DHT naïve offre la latence la plus faible grâce au mécanisme de cache, Chord présente une dégradation modérée grâce à la distribution de la charge, et le centralisé sature rapidement en raison de son point de contention unique.

## 5.2 Débit global du système en fonction de la charge

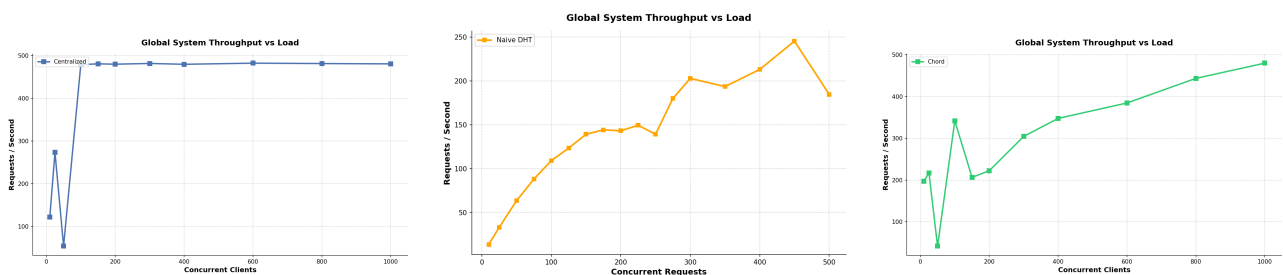


FIGURE 19 – Débit global du système en fonction de la charge.

La Figure 19 montre que Le centralisé converge rapidement vers une stabilité à 480 req/s, maintenu jusqu'à 1000 clients. Ce plafond correspond à la capacité maximale du coordinator, limité par son pool de threads et la sérialisation du routage. Au-delà, l'ajout de clients n'augmente plus le débit — il allonge uniquement les files d'attente, ce qui explique la dégradation de latence observée précédemment.

La DHT naïve présente une croissance initiale du débit qui se stabilise autour de 245 req/s, soit environ la moitié du centralisé. Tant que le nombre de clients reste faible, chaque requête supplémentaire est traitée en parallèle et le débit augmente proportionnellement. Mais à partir d'un certain seuil, les

threads sont tous occupés, les connexions TCP s'accumulent et les ports éphémères s'épuisent — les nouvelles requêtes attendent en file plutôt que d'être traitées immédiatement, ce qui bloque le débit à sa valeur maximale. La limite est ici imposée par les ressources d'exécution de chaque nœud, ce qui explique un plafond plus bas que le centralisé.

Chord présente le comportement le plus favorable à forte charge : le débit continue de progresser jusqu'à 690 req/s à 1000 clients, sans plateau visible. Cette scalabilité s'explique par la distribution de la charge sur les cinq nœuds de l'anneau — contrairement au centralisé dont le coordinator constitue un goulot d'étranglement unique. L'instabilité initiale entre 10 et 100 clients est due à la phase de warm-up de la finger table, qui n'est pas encore complètement stabilisée. Une fois l'anneau convergé, le débit croît de manière régulière et dépasse largement les deux autres architectures sous forte charge.

### 5.3 Efficacité du réseau : nombre moyen de sauts en fonction du nombre de nœuds

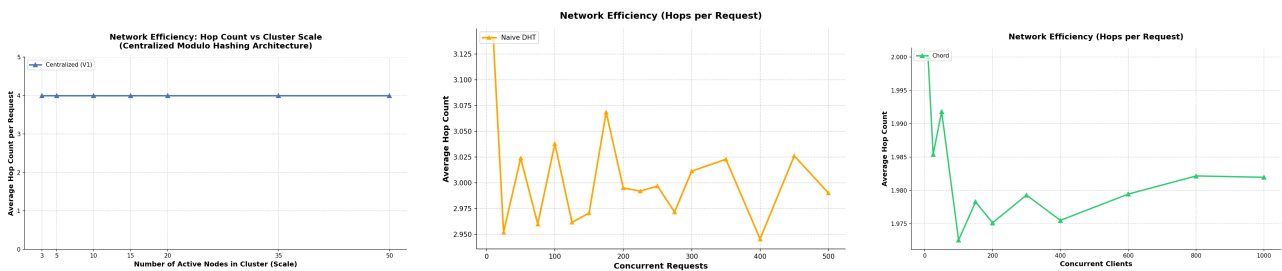


FIGURE 20 – Nombre moyen de sauts.

Dans la Figure 20 le centralisé présente une ligne plate à 4 hops, indépendante du nombre de nœuds actifs. Ce résultat confirme que le chemin d'une requête est entièrement déterminé par la topologie fixe du système : client → coordinator → nœud de stockage → coordinator → client.

La DHT naïve oscille autour de 3 sauts, avec de légères variations selon la charge. Dans une architecture fondée sur l'inondation, le nombre de messages pourrait théoriquement exploser selon le degré des nœuds et la profondeur du TTL. Si ce pire cas n'est pas atteint en pratique, c'est grâce au mécanisme d'élimination des doublons et au *pong caching* qui densifie le réseau et réduit la distance effective entre les paires.

Chord affiche également 3 sauts, stable sur toute la plage 3–50 nœuds. Contrairement aux deux autres architectures, cette stabilité est une propriété de chord : la *finger table* assure un routage en  $O(\log N)$  sauts indépendamment de la charge. La valeur de 3 inclut le saut initial client → nœud d'entrée et est cohérente avec la valeur théorique pour un anneau de taille  $2^8 = 256$ .

## 5.4 Coût du rebalancing en fonction du nombre de nœuds

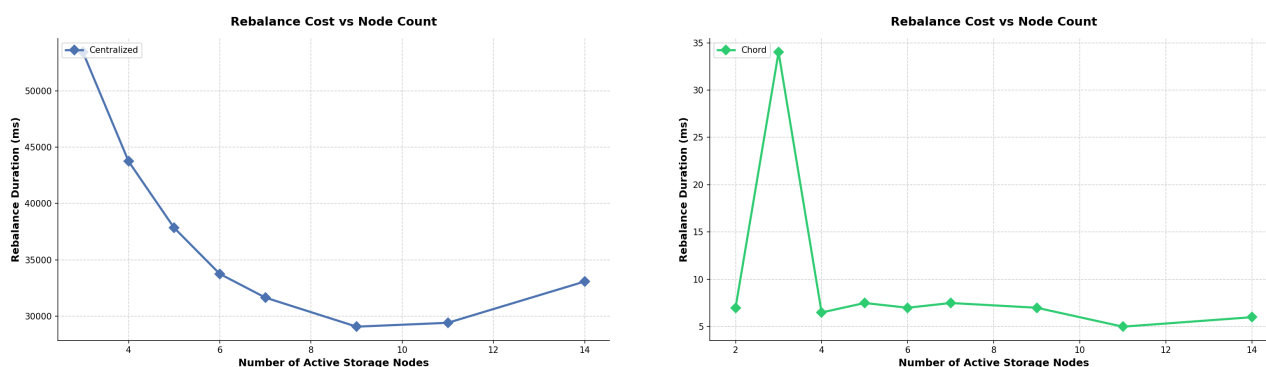


FIGURE 21 – Coût du rebalancing en fonction du nombre de nœuds.

Dans la Figure 21, le centralisé affiche des durées de rebalancing entre 29 et 53 secondes. Lorsque le cluster est petit, chaque nœud survivant reçoit une proportion élevée des clés orphelines, ce qui génère des transferts volumineux. À mesure que le cluster grandit, la charge se répartit sur davantage de nœuds, réduisant la taille de chaque transfert individuel — d'où la décroissance jusqu'à 9 nœuds. La légère remontée au-delà s'explique par l'augmentation du nombre de messages de coordination nécessaires pour orchestrer les transferts entre de nombreux nœuds.

Chord affiche des durées entre 5 et 8 ms pour la quasi-totalité des tailles de cluster, soit un rapport de l'ordre de 1000 par rapport au centralisé. Dans Chord : seul le nœud dont le successeur est mort doit agir, en consultant le premier candidat valide de sa liste de successeurs de secours. La durée ne dépend donc pas de la taille du cluster. Le pic à 330 ms pour 3 nœuds actifs constitue l'unique exception : avec un si petit anneau, le service de stabilisation doit explorer l'ensemble de la finger table avant de trouver un remplaçant joignable, ce qui nécessite plusieurs cycles de 2 secondes.

En résumé, le centralisé effectue un rebalancing global et coûteux, tandis que Chord privilégie une réparation locale rapide.

## 5.5 Distribution des clés avant et après crash

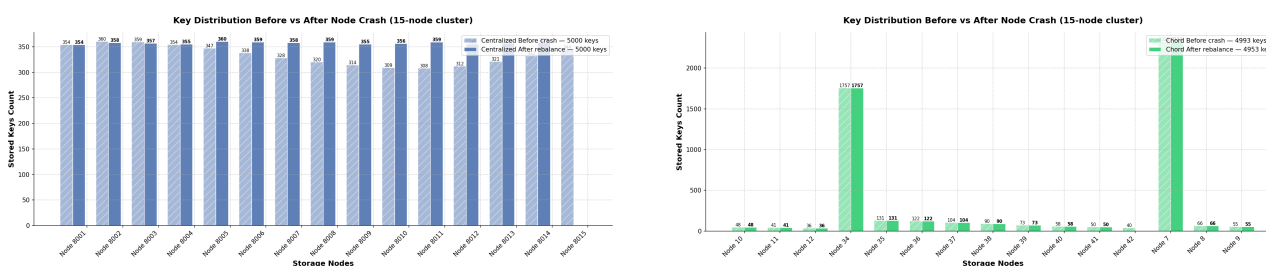


FIGURE 22 – Distribution des clés avant et après crash.

Dans la Figure 22, le Centralisé présente avant le crash une distribution quasi uniforme entre 309 et 360 clés par nœud, ce qui valide l'efficacité du hachage modulo comme mécanisme de répartition. Après le crash du nœud 8015, les clés orphelines sont redistribuées sur les nœuds survivants et le total reste identique à 5000 clés. La distribution demeure homogène, confirmant que le rebalancing node-to-node préserve l'intégralité des données sans perte.

Avec chord, avant le crash, la distribution est fortement hétérogène : les nœuds 34 et 7 concentrent à eux la plupart des clés, tandis que la majorité des nœuds stockent moins de 131 clés. Cette inégalité révèle une limite du *consistent hashing* à petite échelle : avec 15 nœuds sur un anneau de 64 positions, les identifiants ne sont pas régulièrement espacés et certains nœuds héritent de pages de

responsabilité beaucoup plus larges. Après le crash, les nœuds survivants conservent exactement le même nombre de clés qu'avant — mais le total passe de 4993 à 4953 clés, soit une perte définitive de 40 clés appartenant au nœud crashé. Chord répare la topologie de l'anneau, mais sans réplication, les données du nœud mort sont irrécupérables.

## 5.6 Temps de convergence de la topologie pour DHT naïve

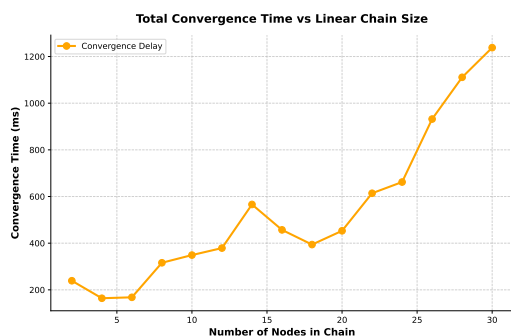


FIGURE 23 – Temps de convergence de la topologie dans une chaîne linéaire de nœuds.

La Figure 23 a été mesurée dans un scénario volontairement défavorable : une chaîne linéaire de  $N$  nœuds dans laquelle chaque pair ne connaît initialement que son voisin immédiat. La courbe obtenue est quasi linéaire — plus le réseau est grand, plus il faut de temps pour que tous les nœuds se connaissent mutuellement.

Ce résultat s'explique simplement : dans une chaîne, l'information doit être relayée de proche en proche via les messages PING/PONG. Un nœud situé à une extrémité ne peut apprendre l'existence d'un nœud à l'autre extrémité qu'après que l'information ait traversé tous les intermédiaires. Le *pong caching* améliore ce processus mais ne change pas sa nature fondamentale.

lement séquentielle. C'est précisément cette limite qui motive le passage aux DHT structurées, dont le routage garantit une découverte efficace indépendamment de la topologie initiale.

## 5.7 Synthèse comparative

Les trois implémentations confirment empiriquement la progression architecturale de ce rapport. Le centralisé offre un comportement prévisible avec 4 hops fixes et un rebalancing sans perte de données, mais son coordinator plafonne le débit à 480 req/s et fait exploser la latence sous forte charge. La DHT naïve stabilise la latence grâce au *reverse path caching* et atteint les latences les plus faibles (50 ms), mais son débit plafonne à 245 req/s et son temps de convergence croît linéairement avec la taille du réseau. Chord présente le meilleur profil à grande échelle : 3 sauts stables de 3 à 50 nœuds, rebalancing local quasi instantané (5–8 ms contre 30–53 secondes pour le centralisé) et débit croissant jusqu'à 690 req/s sans saturation visible.

Ces trois profils illustrent qu'il n'existe pas d'architecture universellement optimale. Le centralisé convient aux contextes où la cohérence et la durabilité des données sont critiques et où le volume de requêtes reste modéré. La DHT naïve trouve sa place dans les scénarios où la découverte rapide de pairs et la faible latence priment, comme la propagation de messages sur un réseau local. Chord est adapté aux systèmes à grande échelle nécessitant un routage efficace et une tolérance aux pannes.

## 6 Conclusion

### 6.1 Usages réels et intérêts pratiques des trois implémentations

#### 6.1.1 Version centralisée

L'architecture centralisée inspirée de *Napster* reste pertinente lorsqu'un point de coordination unique simplifie le système. Ce modèle est facile à développer, superviser et déboguer, ce qui explique sa présence dans de nombreux services modernes : *Google Drive*, *Dropbox* ou *GitHub*<sup>2</sup> et *GitLab*<sup>3</sup> qui reposent sur une forte coordination côté serveur pour gérer les métadonnées, les droits d'accès et la synchronisation. Son principal avantage est la maîtrise complète du système; sa limite est l'existence d'un point critique qui doit être rendu hautement disponible par réplication et redondance.

#### 6.1.2 Version DHT non structurée

Sous sa forme brute, un réseau type Gnutella passe mal à l'échelle pour un stockage distribué global, à cause du coût du flooding et l'instabilité du placement. Mais elle reste pertinente dans des contextes où la simplicité, la robustesse et la découverte rapide priment sur l'optimalité asymptotique.

On retrouve ses idées dans des briques très concrètes : **mDNS**<sup>4</sup> permet à des machines d'un même réseau local de s'annoncer mutuellement sans serveur central, une logique très proche de notre approche.

Dans l'écosystème **libp2p**, les protocoles *Gossipsub*<sup>5</sup> reprennent l'intuition du flooding sous une forme mieux contrôlée pour diffuser messages et blocs dans les blockchains modernes<sup>6</sup>.

Autrement dit, cette approche reste pertinente comme brique de base pour des tâches ciblées : découverte de voisins, diffusion rapide d'information ou maintien d'une connectivité décentralisée — des objectifs complémentaires aux DHT structurées plutôt que concurrents.

#### 6.1.3 Version DHT structurée inspirée de Chord

Chord est utilisée lorsque le routage doit rester décentralisé tout en offrant un espace d'identifiants stable et des tables de routage locales qui permettent d'atteindre le nœud responsable en un nombre logarithmique de sauts.

Dans les systèmes réels, *Amazon Dynamo*<sup>7</sup> a popularisé le *consistent hashing* pour répartir les données et limiter les déplacements lors des changements de topologie, une idée reprise par *Apache Cassandra*<sup>8</sup> et *Riak*<sup>9</sup>. Dans les réseaux pair-à-pair, *BitTorrent Mainline DHT*<sup>10</sup> et *IPFS*<sup>11</sup> s'appuient sur des variantes de *Kademlia*<sup>12</sup> pour localiser efficacement des ressources sans serveur central.

Même si *Chord* reste avant tout un protocole de référence académique, ses principes sont largement présents dans les infrastructures modernes et mieux adaptés que les deux générations précédentes.

### 6.2 Perspectives et optimisations futures

#### 6.2.1 Version Centralisée

- **Coordinators multiples** : remplacer le coordinator unique par plusieurs instances avec répartition de charge pour éliminer le single point of failure et lever le plafond de débit.
- **Réponse directe nœud vers client** : permettre au nœud de stockage de répondre directement au client sans repasser par le coordinator, réduisant le nombre de hops de 4 à 3.
- **Réplication des données** : stocker chaque clé sur plusieurs nœuds pour garantir la disponibilité des données en cas de panne sans perte irrémédiable.

- **Cache coordinator** : mémoriser les résultats des GET récents côté coordinator pour absorber les pics de charge sur les données populaires sans solliciter les nœuds.
- **Consistent hashing côté coordinator** : remplacer la fonction  $\text{hash}(\text{key}) \% N$  du coordinator par un hachage cohérent sur un anneau de taille fixe, afin de limiter les redistributions massives lors des changements de topologie. Contrairement aux générations suivantes qui utilisent le consistent hashing dans un contexte pair-à-pair décentralisé, cette optimisation consiste à l'intégrer directement dans la logique de routage du coordinator, sans modifier l'architecture client-serveur existante.

## 6.2.2 Version DHT non structurée

- **Réplication des données** : optimisation commune aux trois implémentations, déjà détaillée dans la section 6.2.1.
- **TTL adaptatif** : plutôt qu'un plafond fixe de `MAX_HOPS = 10`, un TTL calculé dynamiquement en fonction de la taille estimée du réseau réduirait les messages inutiles dans les petits clusters et garantirait une couverture suffisante dans les grands.
- **Limitation du mesh complet** : le mesh complet implique un coût de maintenance quadratique en  $\Theta(N^2)$ . Une topologie partielle avec un nombre fixe de voisins par nœud, comme dans Kademlia, réduirait ce coût à  $O(\log N)$  tout en conservant une bonne connectivité.

## 6.2.3 Version DHT Chord

- **Réplication des données** : optimisation commune aux trois implémentations, déjà détaillée dans la section 6.2.1.
- **Agrandissement de l'espace d'adressage** : Notre implémentation utilise un anneau de taille  $2^6 = 64$  positions. Avec un nombre croissant de nœuds, les collisions d'identifiants deviennent probables. Passer à  $M = 16$  ou  $M = 32$  bits, en remplaçant le `hashCode` Java par une fonction de hachage cryptographique comme SHA-1 ou SHA-256, améliorerait l'uniformité de la distribution des clés.
- **Service de stabilisation** : La classe `StabilizeService` tourne à intervalle fixe de 2 secondes, indépendamment de l'activité du réseau. Augmenter la fréquence lors d'une période de churn intense et la réduit en période stable, réduirait la consommation de ressources et améliore la réactivité lors des pannes.
- **Liste de successeurs plus longue** : Actuellement, la liste de successeurs de secours est courte. Augmenter cette liste à  $O(\log N)$  entrées renforcerait la tolérance aux pannes simultanées de plusieurs nœuds.

## 6.3 Ouverture : vers des architectures hybrides

L'étude des trois architectures montre que les systèmes distribués modernes combinent rarement un seul paradigme pur. Les piles modernes associent typiquement ces trois approches : un point de contrôle applicatif, une découverte structurée des pairs et une diffusion non structurée des événements.

Les clients Ethereum<sup>6</sup> en sont un bon exemple : le validateur interagit avec un client local, la découverte des pairs s'appuie sur des mécanismes structurés comme *Discv5*<sup>13</sup>, tandis que la diffusion effective de l'état repose sur *Gossipsub*<sup>5</sup>. Les trois générations étudiées dans ce rapport ne sont donc pas de simples étapes pédagogiques — elles constituent les briques fondamentales que les systèmes distribués actuels recombinaient pour obtenir simultanément résilience, cohérence et passage à l'échelle.

# Références

- [1] Klaus WEHRLE, Stefan GÖTZ et Simon RIECHE. “Distributed Hash Tables”. In : *Peer-to-Peer Systems and Applications*. Sous la dir. de Ralf STEINMETZ et Klaus WEHRLE. T. 3485. Lecture Notes in Computer Science. Springer, 2005, p. 79-93. DOI : [10.1007/11530657\\_7](https://doi.org/10.1007/11530657_7). URL : [https://doi.org/10.1007/11530657\\_7](https://doi.org/10.1007/11530657_7).
- [2] GITHUB ENGINEERING. *Introducing DGit*. 2015. URL : <https://github.blog/engineering/architecture-optimization/introducing-dgit/> (visité le 20/05/2026).
- [3] GITLAB DOCS. *Gitaly*. URL : <https://docs.gitlab.com/administration/gitaly/> (visité le 20/05/2026).
- [4] Stuart CHESHIRE et Marc KROCHMAL. *Multicast DNS*. RFC 6762. 2013. DOI : [10.17487/RFC6762](https://doi.org/10.17487/RFC6762). URL : <https://www.rfc-editor.org/rfc/rfc6762>.
- [5] Dimitris VYZOVITIS et al. *GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks*. arXiv preprint arXiv:2007.02754. 2020. URL : <https://arxiv.org/abs/2007.02754>.
- [6] ETHEREUM.ORG. *Networking Layer*. URL : <https://ethereum.org/en/developers/docs/networking-layer/> (visité le 20/05/2026).
- [7] Giuseppe DECANDIA et al. “Dynamo: Amazon’s Highly Available Key-Value Store”. In : *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*. T. 41. SOSP ’07 6. ACM, 2007, p. 205-220. DOI : [10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281). URL : <https://doi.org/10.1145/1294261.1294281>.
- [8] Avinash LAKSHMAN et Prashant MALIK. “Cassandra: A Decentralized Structured Storage System”. In : *ACM SIGOPS Operating Systems Review* 44.2 (2010), p. 35-40. DOI : [10.1145/1773912.1773922](https://doi.org/10.1145/1773912.1773922). URL : <https://doi.org/10.1145/1773912.1773922>.
- [9] RIAK DOCUMENTATION. *Riak KV Concepts*. URL : <https://docs.riak.com/riak/kv/latest/learn/concepts/index.html> (visité le 20/05/2026).
- [10] BITTORRENT ENHANCEMENT PROPOSALS. *BEP 5: DHT Protocol*. URL : [https://www.bittorrent.org/beps/bep\\_0005.html](https://www.bittorrent.org/beps/bep_0005.html) (visité le 20/05/2026).
- [11] IPFS DOCUMENTATION. *Distributed Hash Tables*. URL : <https://docs.ipfs.tech/concepts/dht/> (visité le 20/05/2026).
- [12] Petar MAYMOUNKOV et David MAZIÈRES. “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric”. In : *Peer-to-Peer Systems*. T. 2429. Lecture Notes in Computer Science. Springer, 2002, p. 53-65. DOI : [10.1007/3-540-45748-8\\_5](https://doi.org/10.1007/3-540-45748-8_5). URL : [https://doi.org/10.1007/3-540-45748-8\\_5](https://doi.org/10.1007/3-540-45748-8_5).
- [13] ETHEREUM FOUNDATION. *Discv5 Protocol Specification*. URL : <https://github.com/ethereum/devp2p/blob/master/discv5/discv5.md> (visité le 20/05/2026).
- [14] SCIENCES SORBONNE UNIVERSITÉ. *Fichier: Sciences SU.png*. Logo utilisé sur la page de garde. URL : [https://fr.wikipedia.org/wiki/Fichier:Sciences\\_SU.png](https://fr.wikipedia.org/wiki/Fichier:Sciences_SU.png) (visité le 22/05/2026).